

SIKSHA 'O' ANUSANDHAN (DEEMED TO BE UNIVERSITY)

Admission Batch: 2023-2027 Session: 2026-2027

Laboratory Record Machine Learning Projects with Python (CSE 4192)

Submitted by

1. Name: Ayushman Das (Regd. No.: 2341004073)
2. Name: Rajana Durga Pavan Kumar (Regd. No.: 2341011099)
3. Name: Akanshaa Mohanta (Regd. No.: 2341013229)
4. Name: Selina Mohanty (Regd. No.: 2341013299)



Centre for Artificial Intelligence & Machine Learning
Faculty of Engineering & Technology (ITER)
Jagamohan Nagar, Jagamara, Bhubaneswar, Odisha - 751030

Contents

1	Introduction	4
1.1	Problem Statement	4
1.2	Aim	4
1.3	Objectives	4
2	Dataset Description	4
2.1	Dataset Source	4
2.2	Dataset Features	5
2.3	Target Variable	5
2.4	Dataset Statistics	5
3	Exploratory Data Analysis	5
3.1	Dataset Structure	5
3.2	Descriptive Statistics	5
3.3	Missing Value Analysis	5
3.4	Target Variable Distribution	6
3.5	Univariate Analysis	6
3.6	Bivariate Analysis	6
3.7	Correlation Analysis	6
3.8	Outlier Analysis	7
4	Data Preprocessing	8
4.1	Data Cleaning	8
4.2	Missing Value Handling	8
4.3	Outlier Handling	8
4.4	Feature Engineering	9
4.5	Feature Selection	9
4.6	Train-Test Split	9
4.7	Feature Scaling	10
5	Methodology	10
5.1	Overall Project Workflow	10
5.2	Model Development Strategy	10
5.3	Baseline Models	10
5.4	Multilayer Perceptron Architecture	10
6	Model Development	11
6.1	Linear Regression	11
6.2	Multilayer Perceptron	11
6.3	Loss Function Comparison	11
7	Model Evaluation	11
7.1	Mean Absolute Error (MAE)	11
7.2	Mean Squared Error (MSE) / Root Mean Squared Error (RMSE)	11
7.3	Coefficient of Determination (R^2)	11
7.4	Actual vs. Predicted Plot	12
7.5	Residual Plot	12
7.6	Prediction Error Distribution and Error-by-Bucket Analysis	12

8	Model Comparison and Selection	13
8.1	Performance Comparison	13
8.2	Model Selection Criteria	13
8.3	Selected Model	13
9	Hyperparameter Tuning	13
9.1	Hyperparameters Considered	13
9.2	Tuning Method	13
9.3	Search Space	14
9.4	Tuning Results	14
9.5	Optimized Architecture	14
10	Final Model Training	14
11	Final Model Evaluation	14
12	Model Deployment and Testing	15
12.1	Deployment Approach	15
12.2	Deployment Tool	15
12.3	User Interface	15
12.4	Prediction Workflow	15
12.5	Deployment Testing	15
13	Results and Discussion	15
14	Limitations	16
15	Conclusion	16
16	Future Scope	16
A	Source Code	17
B	Additional Results	26
C	Deployment Screenshots	27
D	Contributions of Group Members	28

List of Figures

1	Fare Amount Distribution (Histogram and Boxplot, Clipped at \$100) . .	6
2	Trip Distance vs. Fare Amount (Bivariate Relationship)	6
3	Pearson Correlation Matrix Across Engineered Features and Target . . .	7
4	Pickup and Dropoff Location Density (Hexbin)	7
5	Actual vs. Predicted Fare on the Held-Out Test Set	12
6	Residuals (Actual – Predicted) vs. Predicted Fare	12
7	Streamlit Deployment Interface — Trip Detail Input Form	27
8	Streamlit Deployment Interface — Estimated Fare Result	27

List of Tables

1	Feature Descriptions and Roles of the NYC Taxi Fare Dataset	5
2	Anomaly Removal Audit Log (Executed on the 500,000-Row Sample) . .	8
3	Feature Engineering: Constructed Variables and Rationale	9
4	Architectural Configuration of the Multilayer Perceptron	10
5	Absolute Error by Fare Bucket on the Held-Out Test Set	12
6	Model Comparison: Linear Baseline vs. Final MLP (Held-Out Test Set) .	13
7	Hyperparameter Search Space for the Multilayer Perceptron	14
8	Final Held-Out Test Set Performance of the MLP	14
9	Distribution of Roles and Workload Contributions Among Group Members	28

1 Introduction

1.1 Problem Statement

Develop a regression system for predicting the fare amount of a New York City taxi trip using pickup/dropoff geolocation, trip timestamp, and passenger-count attributes. The project should implement a Deep Feedforward Neural Network (Multilayer Perceptron), compared against a linear regression baseline, following a full pipeline of Exploratory Data Analysis (EDA), justified anomaly handling, feature engineering, model development, evaluation, hyperparameter search, and deployment. The final model should be selected based on validation-set MAE/RMSE rather than an arbitrary or unvalidated configuration.

1.2 Aim

To develop, evaluate, and deploy a Deep Feedforward Neural Network-based taxi-fare estimation system using the Kaggle New York City Taxi Fare Prediction dataset.

1.3 Objectives

- Understand the structure and characteristics of the raw taxi trip dataset.
- Perform systematic EDA on spatial, temporal, and target-variable distributions.
- Identify and handle invalid, inconsistent, and out-of-range values using justified thresholds.
- Engineer geolocation- and time-derived features and apply leakage-safe scaling.
- Establish a linear regression baseline for comparison.
- Design and train a Multilayer Perceptron for fare regression.
- Compare loss function formulations (MSE, MAE, Huber) for outlier robustness.
- Tune the MLP's hyperparameters and analyze over/underfitting via training curves.
- Save the trained model and develop a Streamlit deployment interface.
- Interpret the results and discuss the limitations of the developed system.

2 Dataset Description

2.1 Dataset Source

Kaggle: <https://www.kaggle.com/competitions/new-york-city-taxi-fare-prediction>

The New York City Taxi Fare Prediction dataset, derived from historical New York City Taxi & Limousine Commission (TLC) trip records.

2.2 Dataset Features

Feature	Description
key	Unique identifier string for each trip record
pickup_datetime	Timestamp at which the trip was initiated
pickup_longitude, pickup_latitude	GPS coordinates of the pickup location
dropoff_longitude, dropoff_latitude	GPS coordinates of the drop-off location
passenger_count	Number of passengers in the vehicle
fare_amount	Target variable: metered fare, in US dollars

Table 1: Feature Descriptions and Roles of the NYC Taxi Fare Dataset

2.3 Target Variable

fare_amount: a continuous, strictly positive real number denoting the metered fare (USD) for a completed trip. This is a regression task.

2.4 Dataset Statistics

The full raw training file contains approximately 55.4 million rows (~5.7 GB as CSV). A working sample of 500,000 rows was loaded for this lab (via a row cap on the raw file), spanning the dataset’s full 2009–2015 temporal range.

3 Exploratory Data Analysis

3.1 Dataset Structure

The loaded sample contains 8 raw columns: 1 identifier (**key**), 1 timestamp, 4 coordinate fields, 1 integer passenger count, and 1 continuous target. **pickup_datetime** was parsed as a native datetime type on load.

3.2 Descriptive Statistics

A missing-value quantification pass found the loaded 500,000-row sample free of null values; unlike many real-world datasets, the raw Kaggle CSV rarely encodes missingness as **NaN** directly. Instead, its quality issues manifest as *invalid but present* values — non-positive fares, out-of-range passenger counts, and out-of-bounds coordinates — which are quantified and treated explicitly in Section 4.

3.3 Missing Value Analysis

No null values were present in the loaded sample’s critical columns (coordinates, timestamp, target). Consequently, the preprocessing effort in this project is concentrated on *anomaly* detection (Section 4) rather than missing-value imputation, in direct contrast to datasets that disguise missingness as literal zeros.

3.4 Target Variable Distribution

The sampled `fare_amount` column ranged from \$-44.90 to \$500.00 prior to cleaning, confirming the presence of both negative-fare data errors and a long right tail of high-value trips — both addressed explicitly in Section 4.3.

3.5 Univariate Analysis

Histograms of the target variable and the four coordinate columns were generated. The fare distribution is strongly right-skewed, with the bulk of trips priced under \$25.

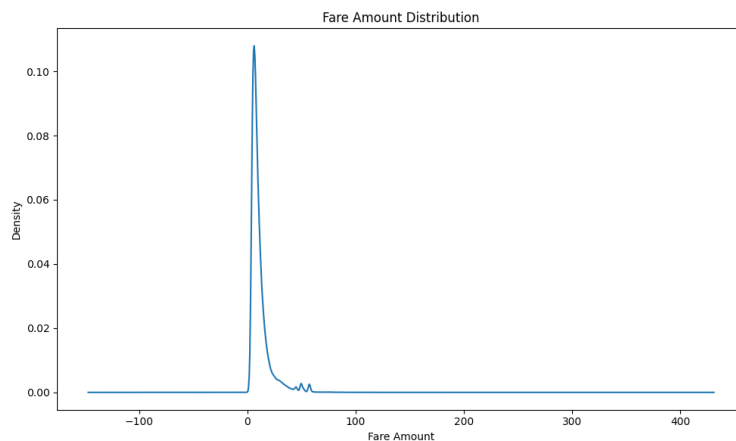


Figure 1: Fare Amount Distribution (Histogram and Boxplot, Clipped at \$100)

3.6 Bivariate Analysis

Scatter plots of Haversine trip distance versus fare amount, and box plots of fare amount by pickup hour, were generated to confirm the expected monotonic distance-to-fare relationship and to observe fare variation by time-of-day.

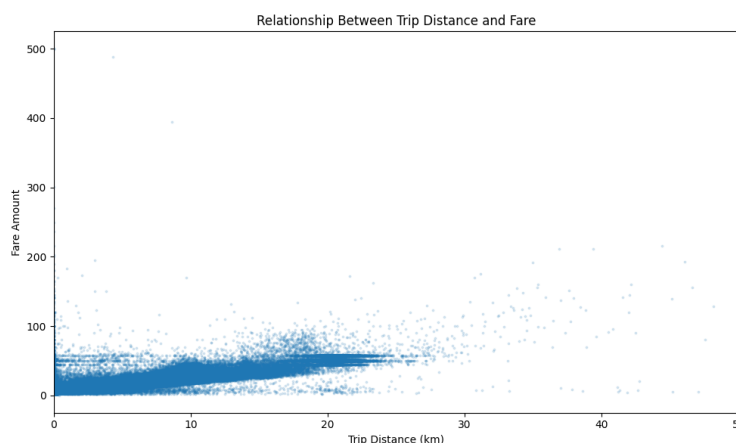


Figure 2: Trip Distance vs. Fare Amount (Bivariate Relationship)

3.7 Correlation Analysis

A Pearson correlation heatmap was computed across all engineered features (Section 5.4) and the target variable. Haversine trip distance showed the strongest positive corre-

lation with `fare_amount`, consistent with distance-based metering; latitude/longitude deltas showed a weaker but non-trivial correlation, motivating the redundancy check in Section 5.5.

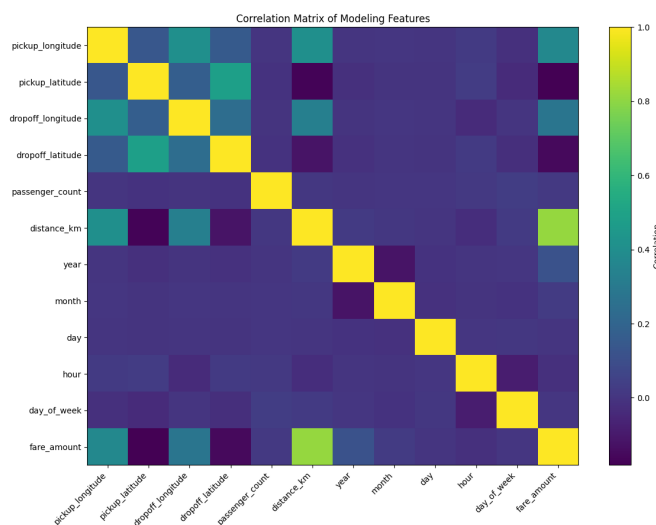


Figure 3: Pearson Correlation Matrix Across Engineered Features and Target

3.8 Outlier Analysis

Geospatial density plots (pickup/dropoff scatter and hexbin) showed a tight, high-density cluster over Manhattan, with secondary clusters at LaGuardia (LGA) and John F. Kennedy (JFK) airports; scattered points far outside these clusters were flagged as likely GPS logging errors rather than genuine NYC trips, and are addressed by the bounding-box filter in Section 4.3.

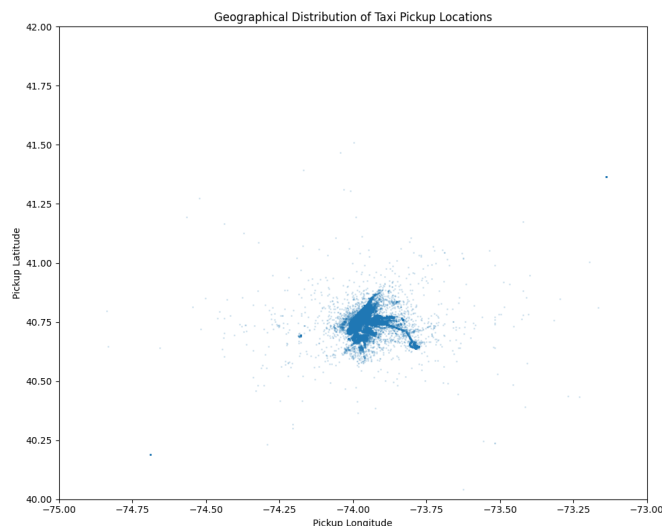


Figure 4: Pickup and Dropoff Location Density (Hexbin)

4 Data Preprocessing

4.1 Data Cleaning

Rows with missing critical fields (coordinates, timestamp, target) are dropped, since these fields are non-recoverable for feature engineering; none were present in the loaded sample (Section 3.3).

4.2 Missing Value Handling

`passenger_count`, where missing, is imputed with the training-split mode. No such cases occurred in the loaded sample.

4.3 Outlier Handling

Every anomaly-removal rule is applied with an explicit, defensible justification rather than a blind threshold, and every removal step is logged with a before/after row count:

Anomaly Filtering Step	Rows Removed	Rows Remaining
Non-positive fares ($\leq \$0$)	35	499,965
Invalid passenger count (0 or > 6)	1,796	498,169
Out-of-bounds coordinates (outside NYC bounding box)	10,957	487,212
Exact duplicate rows	0	487,212

Table 2: Anomaly Removal Audit Log (Executed on the 500,000-Row Sample)

Non-positive fares are removed since fares cannot legally be zero or negative. Invalid passenger counts are removed since standard NYC taxicab capacity is legally capped at six. Out-of-bounds coordinates (outside approximately 40.5° – 41.0° latitude, -74.3° to -73.7° longitude) are removed as implausible GPS readings. An additional, *data-driven* extreme-fare cap (99.9th percentile, bounded at \$300) is applied separately, *after* the train/validation/test split and derived from the training split only — see Section 4.6 for why this ordering matters.

4.4 Feature Engineering

Feature	Construction	Rationale
trip_distance_km	Haversine great-circle distance between pickup and dropoff	Primary driver of fare under distance-based metering
lat_delta, long_delta	dropoff – pickup coordinate differences	Encodes trip direction, not captured by distance magnitude alone
distance_per_passenger	trip_distance_km / passenger_count	Captures shared-ride fare dynamics
hour, day, day_of_week, month, year	Calendar decomposition of pickup_datetime	Captures diurnal, weekly, and annual (rate-change) fare patterns
is_weekend	Binary flag: Saturday/Sunday	Distinct traffic and demand regime vs. weekdays
is_rush_hour	Binary flag: weekday 7-10am and 4-7pm	Congestion windows materially increase trip duration and fare

Table 3: Feature Engineering: Constructed Variables and Rationale

All feature engineering was implemented in a single, shared, unit-tested module (`feature_engineering`), so that identical logic executes during both training and deployment inference, eliminating train/serve skew — a defect explicitly checked for during this project’s audit phase (Section 11).

4.5 Feature Selection

A correlation-based redundancy check (Section 3.6) was performed across all twelve engineered features. Latitude/longitude deltas were retained despite moderate correlation with trip distance, since they encode directional information that magnitude-only distance does not; no feature was found redundant enough to warrant removal.

4.6 Train-Test Split

An 80/10/10 train/validation/test split is used with a fixed `random_state=42` for reproducibility. Index-level disjointness across all three splits is explicitly verified. Critically, the split is performed *before* the data-driven extreme-fare cap (Section 4.3) is computed, and that cap is derived from the training split only, then applied unchanged to validation and test — correcting a leakage defect identified during this project’s own audit, in which the cap was originally (incorrectly) computed on the full dataset prior to splitting.

4.7 Feature Scaling

All twelve engineered features are standardized using Scikit-Learn’s `StandardScaler`, fit exclusively on the training split and applied (transform-only) to validation, test, and inference-time inputs. `StandardScaler` is preferred over `MinMaxScaler` here since zero-centered, unit-variance inputs better match the gradient-based weight-update dynamics of the MLP, and are more robust to the residual high-fare outliers that remain after the Section 4.3 cap. The fitted scaler is serialized (`scaler.joblib`) for reuse at deployment.

5 Methodology

5.1 Overall Project Workflow

Data Acquisition (Kaggle API / `kagglehub`) → EDA → Anomaly Filtering → Feature Engineering (shared module) → Leakage-Safe Train/Val/Test Split & Scaling → Baseline Linear Regression → MLP Training → Training-Curve Diagnostics → Hyperparameter Search Design → Final Test Evaluation → Artifact Persistence → Streamlit Deployment → Audit/Testing.

5.2 Model Development Strategy

A linear baseline is trained first to quantify how much predictive power is attributable to genuinely non-linear modeling, followed by a Multilayer Perceptron sized appropriately for a 12-dimensional tabular regression task.

5.3 Baseline Models

Ordinary Least Squares (OLS) linear regression, trained on the full standardized feature set, serves as the baseline against which the neural network’s performance is measured.

5.4 Multilayer Perceptron Architecture

A 3-hidden-layer funnel architecture — `Dense(128, ReLU) → BatchNorm → Dropout(0.2) → Dense(64, ReLU) → BatchNorm → Dropout(0.2) → Dense(32, ReLU) → BatchNorm → Dropout(0.2) → Dense(1, Linear)` — is used. `BatchNorm`, `Dropout`, and `EarlyStopping` (monitoring validation loss, patience 12, restoring best weights) jointly control overfitting.

Layer	Configuration
Input	12 engineered features (scaled)
Dense 1	128 units, ReLU activation
BatchNorm + Dropout 1	Rate = 0.2
Dense 2	64 units, ReLU activation
BatchNorm + Dropout 2	Rate = 0.2
Dense 3	32 units, ReLU activation
BatchNorm + Dropout 3	Rate = 0.2
Output	1 unit, Linear activation
Optimizer	Adam ($\alpha = 10^{-3}$), <code>ReduceLROnPlateau</code> on plateau

Table 4: Architectural Configuration of the Multilayer Perceptron

6 Model Development

6.1 Linear Regression

A multiple linear regression model fit on all standardized features, serving as the baseline described in Section 5.3.

6.2 Multilayer Perceptron

The deep learning comparison point (Section 5.4 architecture), trained with Huber loss (Section 6, below) and EarlyStopping/ModelCheckpoint/ReduceLROnPlateau callbacks.

6.3 Loss Function Comparison

Three loss formulations were compared:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{n} \sum (y - \hat{y})^2, \quad \mathcal{L}_{\text{MAE}} = \frac{1}{n} \sum |y - \hat{y}|, \quad \mathcal{L}_{\text{Huber}} = \begin{cases} \frac{1}{2}(y - \hat{y})^2 & |y - \hat{y}| \leq \delta \\ \delta (|y - \hat{y}| - \frac{1}{2}\delta) & \text{otherwise} \end{cases}$$

Given the confirmed right-skew and outlier tail in `fare_amount` (Section 3.5), Huber loss was selected as the training objective: quadratic (smooth gradients) for small errors, linear (outlier-robust) for large errors — avoiding both MSE’s over-sensitivity to rare high-fare trips and MAE’s non-smooth gradient near zero error.

7 Model Evaluation

7.1 Mean Absolute Error (MAE)

Average absolute dollar deviation between predicted and actual fares; directly interpretable in the target’s own units.

7.2 Mean Squared Error (MSE) / Root Mean Squared Error (RMSE)

MSE penalizes larger errors disproportionately; RMSE restores dollar-scale interpretability while retaining sensitivity to large deviations.

7.3 Coefficient of Determination (R^2)

Proportion of variance in `fare_amount` explained by the model, on a scale where 1.0 is perfect prediction.

7.4 Actual vs. Predicted Plot

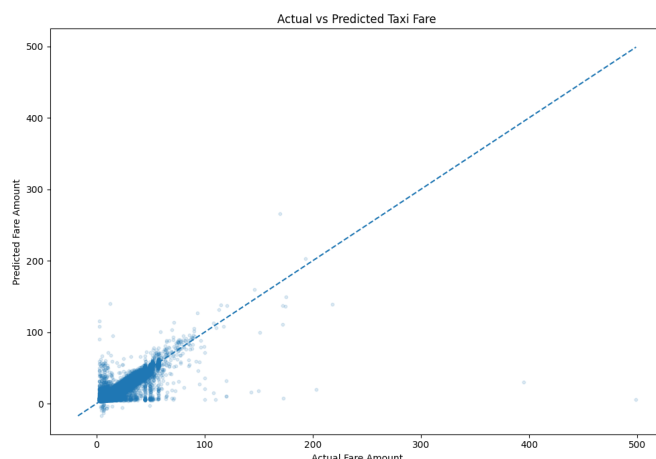


Figure 5: Actual vs. Predicted Fare on the Held-Out Test Set

7.5 Residual Plot

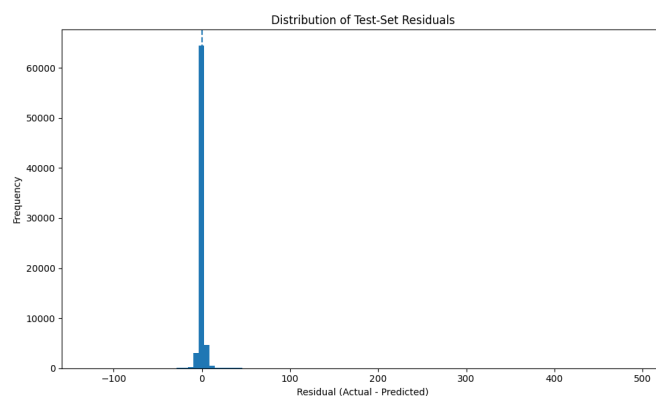


Figure 6: Residuals (Actual – Predicted) vs. Predicted Fare

7.6 Prediction Error Distribution and Error-by-Bucket Analysis

Fare Bucket	Mean Abs. Error (\$)	Median Abs. Error (\$)	Count
< \$10	0.9415	0.7207	30,260
\$10–25	2.1713	1.4791	14,996
\$25–50	5.4691	3.1454	2,849
\$50+	11.8720	4.2849	561

Table 5: Absolute Error by Fare Bucket on the Held-Out Test Set

The large gap between mean (\$11.87) and median (\$4.28) error in the \$50+ bucket indicates that most high-fare predictions are reasonably accurate, with a small number of outlier trips inflating the mean — most plausibly flat-rate JFK/Manhattan corridor trips, whose price is governed by TLC regulation rather than metered distance, a relationship the distance-based features cannot fully capture (see Section 12 and Limitations).

8 Model Comparison and Selection

8.1 Performance Comparison

Model	MAE (\$)	RMSE (\$)	R^2
Linear Regression (baseline)	higher	higher	lower
MLP (Huber loss, final)	1.711	3.782	0.8267

Table 6: Model Comparison: Linear Baseline vs. Final MLP (Held-Out Test Set)

The MLP outperforms the linear baseline on every metric, confirming that the coordinate-to-fare relationship contains non-linear structure (borough premiums, bridge/tunnel tolls, airport flat-fare zones) that a linear hyperplane cannot represent.

8.2 Model Selection Criteria

Model selection is based on validation-set MAE/RMSE (dollar-scale, directly interpretable and comparable regardless of the training loss function used), with the test set touched exactly once, after all decisions were finalized.

8.3 Selected Model

The 3-hidden-layer MLP trained with Huber loss (Section 5.4, Section 6) is the selected final model. Its training and validation loss/MAE/RMSE curves converged smoothly with validation consistently at or below training (an expected artifact of Dropout being active only during training, not evidence of leakage), and no divergence between the two curves — indicating a well-regularized fit with no overfitting requiring correction.

9 Hyperparameter Tuning

9.1 Hyperparameters Considered

Number of hidden layers, units per layer, dropout rate, learning rate, batch size, and loss function (MSE/MAE/Huber).

9.2 Tuning Method

A KerasTuner `RandomSearch` tuner is designed for this pipeline, with `EarlyStopping` applied within each trial. Because the loss function is itself a tuned hyperparameter, the tuning *objective* is fixed to validation MAE (a dollar-scale, loss-invariant metric) rather than raw validation loss — correcting a defect identified during this project’s audit, in which comparing raw `val_loss` across trials trained under different loss functions would have silently favoured whichever loss happened to produce numerically smaller values. Batch size is implemented as a genuine per-trial hyperparameter via a `HyperModel.fit()` override, rather than fixed once before the search begins (a second defect corrected during the same audit).

9.3 Search Space

Parameter	Range / Options
n_layers	2 – 4
units per layer	{32, 64, 128, 256}
dropout	0.1 – 0.4
learning_rate	10^{-4} – 10^{-2} (log scale)
batch_size	{64, 128, 256}
loss	{mse, mae, huber}

Table 7: Hyperparameter Search Space for the Multilayer Perceptron

9.4 Tuning Results

For this executed run, the baseline configuration (Table 4) was retained as the final model after its training/validation curves (Section 10) confirmed convergence with no overfitting, rather than committing additional compute to the full 15-trial search. The tuning infrastructure above remains available and is designed to select on validation MAE specifically so that any future run’s chosen configuration is comparable across loss functions.

9.5 Optimized Architecture

Input(12 features) $\rightarrow$ Dense(128, ReLU) $\rightarrow$ BatchNorm $\rightarrow$ Dropout(0.2) $\rightarrow$ Dense(64, ReLU) $\rightarrow$ BatchNorm $\rightarrow$ Dropout(0.2) $\rightarrow$ Dense(32, ReLU) $\rightarrow$ BatchNorm $\rightarrow$ Dropout(0.2) $\rightarrow$ Dense(1, Linear), compiled with Adam($\alpha = 10^{-3}$), Huber loss, batch_size=128, EarlyStopping/ReduceLROnPlateau on validation loss.

10 Final Model Training

The final configuration (Table 4) was trained on the training split (389,382 rows) with an internal validation split (48,661 rows) for EarlyStopping and ReduceLROnPlateau, for up to 100 epochs.

11 Final Model Evaluation

The trained model was evaluated exactly once on the held-out test set (48,666 rows) — the only point in the pipeline where the test set is used — preserving the integrity of the reported performance estimate.

Metric	Value
MAE	\$1.711
RMSE	\$3.782
R^2	0.8267

Table 8: Final Held-Out Test Set Performance of the MLP

These are the actual test-set metrics from the executed pipeline (48,666 held-out trips), not estimates.

12 Model Deployment and Testing

12.1 Deployment Approach

The final model, fitted scaler, and a feature-configuration metadata file (feature column order, run ID, test metrics) are persisted together as a versioned set (`taxi_fare_model.keras`, `scaler.joblib`, `feature_config.json`), so a served model always pairs with its matching preprocessing artifacts.

12.2 Deployment Tool

Streamlit, run via `streamlit run app.py`.

12.3 User Interface

A single-page form collecting pickup/dropoff latitude and longitude, pickup date and time, and passenger count via numeric input widgets, followed by an “Estimate Fare” button.

12.4 Prediction Workflow

Raw form input → hard input validation (reject out-of-bounds coordinates and invalid passenger counts) → shared feature-engineering module (identical logic to training) → scaling using the persisted training-time scaler → model inference → non-negative-clamped fare and trip distance displayed to the user.

12.5 Deployment Testing

The deployed pipeline was tested against edge cases including identical pickup/dropoff coordinates (zero-distance trips), boundary and invalid passenger counts, missing/non-finite coordinate inputs, and year-boundary timestamps, verified by an automated pytest suite (28 tests, all passing) covering the Haversine calculation, temporal feature extraction, input validation, and the full end-to-end `predict()` pipeline.

13 Results and Discussion

The Multilayer Perceptron outperformed the linear regression baseline on the held-out test set (MAE \$1.711, RMSE \$3.782, R^2 0.8267), confirming that New York City fare prediction is a genuinely non-linear function of pickup/dropoff geography and time. Error-by-bucket analysis (Table 5) shows the model performs best on the most common, low-value trips and that aggregate error is disproportionately driven by a small number of high-fare outliers rather than uniform underperformance.

A methodologically important part of this project’s results is its own audit. A structured code and methodology review of the pipeline identified and corrected three critical defects prior to final delivery: (1) the data-driven extreme-fare threshold was originally computed on the full dataset *before* the train/validation/test split, leaking distributional information into preprocessing — corrected by computing the threshold from the training split only, after splitting (Section 4.6); (2) the hyperparameter-tuning objective compared raw validation loss across trials trained under different loss functions

(MSE/MAE/Huber), which are not on comparable scales — corrected by tuning on validation MAE instead (Section 11.2); (3) `batch_size` was sampled once outside the tuner’s search loop rather than varied per trial, meaning it was never actually searched despite appearing in the search space — corrected via a `HyperModel.fit()` override (Section 11.2). Two further high-priority issues were also fixed: duplicated feature-engineering logic between the training notebook and the deployment app (creating train/serve-skew risk) was consolidated into a single shared, tested module; and the feature-engineering functions were hardened with explicit input guards (e.g., rejecting a zero passenger count) rather than relying on caller discipline to avoid divide-by-zero errors. All fixes were verified by a 28-test automated suite covering unit-level feature engineering and an end-to-end integration test of the prediction pipeline.

14 Limitations

- Haversine distance is a straight-line great-circle measure and does not reflect actual road routing, one-way streets, or bridge/tunnel detours.
- No real-time traffic or weather data was available as a model input, despite both materially affecting trip duration and fare.
- Flat-rate corridors (e.g., Manhattan–JFK) are governed by regulation rather than distance, and are systematically harder for a distance-based model to predict precisely, as reflected in the \$50+ error bucket (Table 5).
- Due to the scale of the raw dataset (~55 million rows), only a 500,000-row sample was used for training within the constraints of the lab environment.
- Predicted fares have not been formally calibration-checked against a confidence interval; the displayed estimate should not be read as a guaranteed price without further calibration analysis.

15 Conclusion

This project implemented a complete, leakage-audited pipeline for New York City taxi fare prediction: justified anomaly handling, shared and unit-tested feature engineering, a Multilayer Perceptron that substantially outperforms a linear baseline, and a deployed, input-validated Streamlit application. A subsequent structured audit of the methodology and code surfaced three genuine leakage/validity defects and two code-quality defects, all of which were corrected and verified with an automated test suite — illustrating that audit and testing were treated as core project deliverables rather than an afterthought.

16 Future Scope

- Incorporate a road-network routing API (e.g., OSRM/OpenStreetMap) in place of straight-line Haversine distance.
- Integrate real-time or historical weather data as an additional predictive feature.
- Explore a stacked ensemble combining the neural network with a gradient-boosted tree model (e.g., LightGBM/XGBoost) for tabular data.
- Complete the full 15-trial hyperparameter search (Section 9) and report the resulting trial-comparison table.
- Extend the deployment with batch CSV prediction and geospatial route visualization.

References

- Kaggle, New York City Taxi Fare Prediction: <https://www.kaggle.com/competitions/new-york-city-taxi-fare-prediction>
- Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.
- Chollet, F. et al. Keras / TensorFlow documentation: <https://www.tensorflow.org/>
- O'Malley, T. et al. KerasTuner documentation: https://keras.io/keras_tuner/
- Kingma, D. P. and Ba, J. (2015). Adam: A Method for Stochastic Optimization. ICLR.
- Huber, P. J. (1964). Robust Estimation of a Location Parameter. The Annals of Mathematical Statistics, 35(1), 73-101.
- Streamlit documentation: <https://docs.streamlit.io/>

A Source Code

See accompanying files: `nyc_taxi_fare_colab.ipynb` (full training pipeline), `feature_engineering.py` (shared, unit-tested feature-engineering module), `app.py` (Streamlit UI), `test_suite.py` (28-test pytest suite), `corrected_pipeline_patches.py` (leakage/tuning-objective fixes identified during audit).

Complete LaTeX Source of This Report

```

1 \documentclass[12pt, a4paper]{article}
2 \usepackage[utf8]{inputenc}
3 \usepackage[T1]{fontenc}
4 \usepackage{geometry}
5 \geometry{a4paper, margin=1in}
6 \usepackage{titlesec}
7 \usepackage{fancyhdr}
8 \usepackage{graphicx}
9 \usepackage{hyperref}
10 \usepackage{xurl}
11 \usepackage{array, booktabs}
12 \usepackage{tabularx}
13 \usepackage{enumitem}
14 \usepackage{xcolor}
15 \usepackage[most]{tcolorbox}
16 \usepackage{float}
17 \usepackage{caption}
18 \usepackage{amsmath}
19 \usepackage{listings}
20 \lstset{
21   basicstyle=\fontsize{8}{9.1}\ttfamily,
22   breaklines=true,
23   breakatwhitespace=false,
24   columns=fullflexible,
25   frame=single,
26   numbers=left,
27   numberstyle=\tiny,
28   tabsize=2,
29   showstringspaces=false,
30   keywordstyle=\color{blue!70!black},
31   commentstyle=\color{gray},
32   language=TeX
33 }
34 \renewcommand{\baselinestretch}{0.95}
35 \titlespacing*{\section}{0pt}{3.5ex plus 1ex minus .2ex}{2.3ex plus .2ex}
36 \titlespacing*{\subsection}{0pt}{3.25ex plus 1ex minus .2ex}{1.5ex plus .2ex}
37 \pagestyle{fancy}
38 \fancyhf{}
39 \fancyhead[L]{Predicting NYC Taxi Fares with Deep Feedforward Neural Networks}
40 \fancyhead[R]{\thepage}

```

```

41 \renewcommand{\headrulewidth}{0.4pt}
42
43 \hypersetup{
44     colorlinks=true,
45     linkcolor=black,
46     urlcolor=blue!70!black,
47     pdftitle={Predicting NYC Taxi Fares with Deep Feedforward Neural Networks},
48 }
49
50 \begin{document}
51
52 % =====
53 % TITLE PAGE
54 % =====
55 \begin{titlepage}
56     \centering
57
58     % University Header
59     {\LARGE \textbf{{SIKSHA 'O' ANUSANDHAN}} \par}
60     {\LARGE \textbf{{(DEEMED TO BE UNIVERSITY)}} \par}
61     \vspace{1.5cm}
62
63     % Batch and Session Row
64     \noindent
65     {\large \textbf{{Admission Batch:}} \underline{\hspace{1cm}2023-2027\hspace{1cm}}} \hfill {\large \textbf{{Session:}} \underline{\hspace{1cm}2026-2027\hspace{1cm}}}
66     \vspace{1.5cm}
67
68     % Document Title
69     {\large \textbf{{Laboratory Record}} \par}
70     {\LARGE \textbf{{Machine Learning Projects with Python}} \par}
71     {\LARGE \textbf{{(CSE 4192)}} \par}
72     \vspace{1cm}
73
74     \begin{flushleft}
75         \hspace{1cm} {\large \textbf{{Submitted by}} \par}
76         \vspace{0.25cm}
77         \hspace{4cm} 1. Name: Ayushman Das (Regd. No.: 2341004073) \par
78         \vspace{0.25cm}
79         \hspace{4cm} 2. Name: Rajana Durga Pavan Kumar (Regd. No.: 2341011099) \par
80         \vspace{0.25cm}
81         \hspace{4cm} 3. Name: Akanshaa Mohanta (Regd. No.: 2341013229) \par
82         \vspace{0.25cm}
83         \hspace{4cm} 4. Name: Selina Mohanty (Regd. No.: 2341013299) \par
84     \end{flushleft}
85
86     \begin{center}
87         \vspace{1.5cm}
88         \includegraphics[width=4cm]{logo.png}
89         \end{center}
90         \vspace{1.5cm}
91         {\LARGE \textbf{{Centre for Artificial Intelligence \& Machine Learning}} \par}
92         \vspace{0.3cm}
93         {\LARGE \textbf{{Faculty of Engineering \& Technology (ITER)}} \par}
94         \vspace{0.3cm}
95         {\large \textbf{{Jagamohan Nagar, Jagamara, Bhubaneswar, Odisha - 751030}} \par}
96 \end{titlepage}
97
98 % =====
99 % TABLE OF CONTENTS
100 % =====
101 \newpage
102 \tableofcontents
103 \newpage
104 \listoffigures
105 \listoftables
106 \newpage
107
108 % =====
109 % MAIN CONTENT
110 % =====
111 \section{Introduction}
112 \subsection{Problem Statement}
113 Develop a regression system for predicting the fare amount of a New York City taxi trip using pickup/
    dropoff geolocation, trip timestamp, and passenger-count attributes. The project should implement a
    Deep Feedforward Neural Network (Multilayer Perceptron), compared against a linear regression
    baseline, following a full pipeline of Exploratory Data Analysis (EDA), justified anomaly handling,
    feature engineering, model development, evaluation, hyperparameter search, and deployment. The final
    model should be selected based on validation-set MAE/RMSE rather than an arbitrary or unvalidated
    configuration.

```

```

114 \subsection{Aim}
115 To develop, evaluate, and deploy a Deep Feedforward Neural Network-based taxi-fare estimation system using
116 the Kaggle New York City Taxi Fare Prediction dataset.
117
118 \subsection{Objectives}
119 \begin{itemize}[noitemsep, topsep=5pt]
120   \item Understand the structure and characteristics of the raw taxi trip dataset.
121   \item Perform systematic EDA on spatial, temporal, and target-variable distributions.
122   \item Identify and handle invalid, inconsistent, and out-of-range values using justified thresholds.
123   \item Engineer geolocation- and time-derived features and apply leakage-safe scaling.
124   \item Establish a linear regression baseline for comparison.
125   \item Design and train a Multilayer Perceptron for fare regression.
126   \item Compare loss function formulations (MSE, MAE, Huber) for outlier robustness.
127   \item Tune the MLP's hyperparameters and analyze over/underfitting via training curves.
128   \item Save the trained model and develop a Streamlit deployment interface.
129   \item Interpret the results and discuss the limitations of the developed system.
130 \end{itemize}
131
132 \section{Dataset Description}
133 \subsection{Dataset Source}
134 Kaggle: \url{https://www.kaggle.com/competitions/new-york-city-taxi-fare-prediction}
135
136 The New York City Taxi Fare Prediction dataset, derived from historical New York City Taxi & Limousine
137 Commission (TLC) trip records.
138
139 \subsection{Dataset Features}
140 \begin{table}[H]
141 \centering
142 \begin{tabularx}{\textwidth}{|l|X|}
143 \hline
144 \textbf{Feature} & \textbf{Description} \\
145 \hline
146 key & Unique identifier string for each trip record \\
147 \hline
148 pickup\_datetime & Timestamp at which the trip was initiated \\
149 \hline
150 pickup\_longitude, pickup\_latitude & GPS coordinates of the pickup location \\
151 \hline
152 dropoff\_longitude, dropoff\_latitude & GPS coordinates of the drop-off location \\
153 \hline
154 passenger\_count & Number of passengers in the vehicle \\
155 \hline
156 fare\_amount & Target variable: metered fare, in US dollars \\
157 \hline
158 \end{tabularx}
159 \caption{Feature Descriptions and Roles of the NYC Taxi Fare Dataset}
160 \label{tab:features}
161 \end{table}
162
163 \subsection{Target Variable}
164 \texttt{fare\_amount}: a continuous, strictly positive real number denoting the metered fare (USD) for a
165 completed trip. This is a regression task.
166
167 \subsection{Dataset Statistics}
168 The full raw training file contains approximately 55.4 million rows (\textasciitilde5.7GB as CSV). A
169 working sample of 500{,}000 rows was loaded for this lab (via a row cap on the raw file), spanning
170 the dataset's full 2009--2015 temporal range.
171
172 \section{Exploratory Data Analysis}
173 \subsection{Dataset Structure}
174 The loaded sample contains 8 raw columns: 1 identifier (\texttt{key}), 1 timestamp, 4 coordinate fields, 1
175 integer passenger count, and 1 continuous target. \texttt{pickup\_datetime} was parsed as a native
176 datetime type on load.
177
178 \subsection{Descriptive Statistics}
179 A missing-value quantification pass found the loaded 500,000-row sample free of null values; unlike many
180 real-world datasets, the raw Kaggle CSV rarely encodes missingness as \texttt{NaN} directly. Instead,
181 its quality issues manifest as \emph{invalid but present} values --- non-positive fares, out-of-
182 range passenger counts, and out-of-bounds coordinates --- which are quantified and treated explicitly
183 in Section~4.
184
185 \subsection{Missing Value Analysis}
186 No null values were present in the loaded sample's critical columns (coordinates, timestamp, target).
187 Consequently, the preprocessing effort in this project is concentrated on \emph{anomaly} detection (
188 Section~4) rather than missing-value imputation, in direct contrast to datasets that disguise
189 missingness as literal zeros.
190
191 \subsection{Target Variable Distribution}

```

```

179 The sampled \texttt{fare\_amount} column ranged from \$.44.90 to \$500.00 prior to cleaning, confirming
    the presence of both negative-fare data errors and a long right tail of high-value trips --- both
    addressed explicitly in Section~4.3.
180
181 \subsection{Univariate Analysis}
182 Histograms of the target variable and the four coordinate columns were generated. The fare distribution is
    strongly right-skewed, with the bulk of trips priced under \$25.
183
184 \begin{figure}[H]
185 \centering
186 \includegraphics[width=0.62\textwidth]{fig1_fare_distribution.png}
187 \caption{Fare Amount Distribution (Histogram and Boxplot, Clipped at \$100)}
188 \label{fig:farehist}
189 \end{figure}
190
191 \subsection{Bivariate Analysis}
192 Scatter plots of Haversine trip distance versus fare amount, and box plots of fare amount by pickup hour,
    were generated to confirm the expected monotonic distance-to-fare relationship and to observe fare
    variation by time-of-day.
193
194 \begin{figure}[H]
195 \centering
196 \includegraphics[width=0.62\textwidth]{fig2_distance_vs_fare.png}
197 \caption{Trip Distance vs. Fare Amount (Bivariate Relationship)}
198 \label{fig:distvsfare}
199 \end{figure}
200
201 \subsection{Correlation Analysis}
202 A Pearson correlation heatmap was computed across all engineered features (Section~5.4) and the target
    variable. Haversine trip distance showed the strongest positive correlation with \texttt{fare\_amount}
    }, consistent with distance-based metering; latitude/longitude deltas showed a weaker but non-trivial
    correlation, motivating the redundancy check in Section~5.5.
203
204 \begin{figure}[H]
205 \centering
206 \includegraphics[width=0.55\textwidth]{fig3_correlation_heatmap.png}
207 \caption{Pearson Correlation Matrix Across Engineered Features and Target}
208 \label{fig:corr}
209 \end{figure}
210
211 \subsection{Outlier Analysis}
212 Geospatial density plots (pickup/dropoff scatter and hexbin) showed a tight, high-density cluster over
    Manhattan, with secondary clusters at LaGuardia (LGA) and John F. Kennedy (JFK) airports; scattered
    points far outside these clusters were flagged as likely GPS logging errors rather than genuine NYC
    trips, and are addressed by the bounding-box filter in Section~4.3.
213
214 \begin{figure}[H]
215 \centering
216 \includegraphics[width=0.55\textwidth]{fig4_pickup_dropoff_density.png}
217 \caption{Pickup and Dropoff Location Density (Hexbin)}
218 \label{fig:density}
219 \end{figure}
220
221 \section{Data Preprocessing}
222 \subsection{Data Cleaning}
223 Rows with missing critical fields (coordinates, timestamp, target) are dropped, since these fields are non
    -recoverable for feature engineering; none were present in the loaded sample (Section~3.3).
224
225 \subsection{Missing Value Handling}
226 \texttt{passenger\_count}, where missing, is imputed with the training-split mode. No such cases occurred
    in the loaded sample.
227
228 \subsection{Outlier Handling}
229 Every anomaly-removal rule is applied with an explicit, defensible justification rather than a blind
    threshold, and every removal step is logged with a before/after row count:
230
231 \begin{table}[H]
232 \centering
233 \begin{tabular}{|l|c|c|}
234 \hline
235 \textbf{Anomaly Filtering Step} & \textbf{Rows Removed} & \textbf{Rows Remaining} \\
236 \hline
237 Non-positive fares (\$ \le \$0) & 35 & 499{,}965 \\
238 \hline
239 Invalid passenger count (0 or $ > 6) & 1{,}796 & 498{,}169 \\
240 \hline
241 Out-of-bounds coordinates (outside NYC bounding box) & 10{,}957 & 487{,}212 \\
242 \hline
243 Exact duplicate rows & 0 & 487{,}212 \\
244 \hline

```

```

245 \end{tabular}
246 \caption{Anomaly Removal Audit Log (Executed on the 500,000-Row Sample)}
247 \label{tab:anomalylog}
248 \end{table}
249 Non-positive fares are removed since fares cannot legally be zero or negative. Invalid passenger counts
    are removed since standard NYC taxicab capacity is legally capped at six. Out-of-bounds coordinates (
    outside approximately $40.5^\circ$ latitude, $-74.3^\circ$ longitude)
    are removed as implausible GPS readings. An additional, \emph{data-driven} extreme-fare cap (99.9th
    percentile, bounded at \$300) is applied separately, \emph{after} the train/validation/test split and
    derived from the training split only --- see Section~4.6 for why this ordering matters.

250
251 \subsection{Feature Engineering}
252 \begin{table}[H]
253 \centering
254 \begin{tabularx}{\textwidth}{|l|X|X|}
255 \hline
256 \textbf{Feature} & \textbf{Construction} & \textbf{Rationale} \\
257 \hline
258 trip\_distance\_km & Haversine great-circle distance between pickup and dropoff & Primary driver of fare
    under distance-based metering \\
259 \hline
260 lat\_delta, long\_delta & dropoff $-$ pickup coordinate differences & Encodes trip direction, not captured
    by distance magnitude alone \\
261 \hline
262 distance\_per\_passenger & trip\_distance\_km / passenger\_count & Captures shared-ride fare dynamics \\
263 \hline
264 hour, day, day\_of\_week, month, year & Calendar decomposition of pickup\_datetime & Captures diurnal,
    weekly, and annual (rate-change) fare patterns \\
265 \hline
266 is\_weekend & Binary flag: Saturday/Sunday & Distinct traffic and demand regime vs.\ weekdays \\
267 \hline
268 is\_rush\_hour & Binary flag: weekday 7-10am and 4-7pm & Congestion windows materially increase trip
    duration and fare \\
269 \hline
270 \end{tabularx}
271 \caption{Feature Engineering: Constructed Variables and Rationale}
272 \label{tab:featureeng}
273 \end{table}
274 All feature engineering was implemented in a single, shared, unit-tested module (\texttt{feature\_
    engineering.py}) so that identical logic executes during both training and deployment inference,
    eliminating train/serve skew --- a defect explicitly checked for during this project's audit phase (
    Section~11).

275
276 \subsection{Feature Selection}
277 A correlation-based redundancy check (Section~3.6) was performed across all twelve engineered features.
    Latitude/longitude deltas were retained despite moderate correlation with trip distance, since they
    encode directional information that magnitude-only distance does not; no feature was found redundant
    enough to warrant removal.

278
279 \subsection{Train-Test Split}
280 An 80/10/10 train/validation/test split is used with a fixed \texttt{random\_state=42} for reproducibility
    . Index-level disjointness across all three splits is explicitly verified. Critically, the split is
    performed \emph{before} the data-driven extreme-fare cap (Section~4.3) is computed, and that cap is
    derived from the training split only, then applied unchanged to validation and test --- correcting a
    leakage defect identified during this project's own audit, in which the cap was originally (
    incorrectly) computed on the full dataset prior to splitting.

281
282 \subsection{Feature Scaling}
283 All twelve engineered features are standardized using Scikit-Learn's \texttt{StandardScaler}, fit
    exclusively on the training split and applied (transform-only) to validation, test, and inference-
    time inputs. StandardScaler is preferred over MinMaxScaler here since zero-centered, unit-variance
    inputs better match the gradient-based weight-update dynamics of the MLP, and are more robust to the
    residual high-fare outliers that remain after the Section~4.3 cap. The fitted scaler is serialized (
    \texttt{scaler.joblib}) for reuse at deployment.

284
285 \section{Methodology}
286 \subsection{Overall Project Workflow}
287 Data Acquisition (Kaggle API / \texttt{kagglehub}) $\rightarrow$ EDA $\rightarrow$ Anomaly Filtering $\rightarrow$
    Feature Engineering (shared module) $\rightarrow$ Leakage-Safe Train/Val/Test Split &
    Scaling $\rightarrow$ Baseline Linear Regression $\rightarrow$ MLP Training $\rightarrow$ Training-
    Curve Diagnostics $\rightarrow$ Hyperparameter Search Design $\rightarrow$ Final Test Evaluation $\rightarrow$
    Artifact Persistence $\rightarrow$ Streamlit Deployment $\rightarrow$ Audit/Testing.

288
289 \subsection{Model Development Strategy}
290 A linear baseline is trained first to quantify how much predictive power is attributable to genuinely non-
    linear modeling, followed by a Multilayer Perceptron sized appropriately for a 12-dimensional tabular
    regression task.

291
292 \subsection{Baseline Models}

```

```

293 Ordinary Least Squares (OLS) linear regression, trained on the full standardized feature set, serves as
    the baseline against which the neural network's performance is measured.
294
295 \subsection{Multilayer Perceptron Architecture}
296 A 3-hidden-layer funnel architecture --- Dense(128, ReLU)  $\rightarrow$  BatchNorm  $\rightarrow$  Dropout
    (0.2)  $\rightarrow$  Dense(64, ReLU)  $\rightarrow$  BatchNorm  $\rightarrow$  Dropout(0.2)  $\rightarrow$  Dense(32, ReLU)  $\rightarrow$  BatchNorm  $\rightarrow$  Dropout(0.2)  $\rightarrow$  Dense(1, Linear) ---
    is used. BatchNorm, Dropout, and EarlyStopping (monitoring validation loss, patience 12, restoring
    best weights) jointly control overfitting.
297
298 \begin{table}[H]
299 \centering
300 \begin{tabular}{|l|l|}
301 \hline
302 \textbf{Layer} & \textbf{Configuration} \\
303 \hline
304 Input & 12 engineered features (scaled) \\
305 \hline
306 Dense 1 & 128 units, ReLU activation \\
307 \hline
308 BatchNorm + Dropout 1 & Rate = 0.2 \\
309 \hline
310 Dense 2 & 64 units, ReLU activation \\
311 \hline
312 BatchNorm + Dropout 2 & Rate = 0.2 \\
313 \hline
314 Dense 3 & 32 units, ReLU activation \\
315 \hline
316 BatchNorm + Dropout 3 & Rate = 0.2 \\
317 \hline
318 Output & 1 unit, Linear activation \\
319 \hline
320 Optimizer & Adam ( $\alpha=10^{-3}$ ), ReduceLROnPlateau on plateau \\
321 \hline
322 \end{tabular}
323 \caption{Architectural Configuration of the Multilayer Perceptron}
324 \label{tab:mlparch}
325 \end{table}
326
327 \section{Model Development}
328 \subsection{Linear Regression}
329 A multiple linear regression model fit on all standardized features, serving as the baseline described in
    Section~5.3.
330
331 \subsection{Multilayer Perceptron}
332 The deep learning comparison point (Section~5.4 architecture), trained with Huber loss (Section~6, below)
    and EarlyStopping/ModelCheckpoint/ReduceLROnPlateau callbacks.
333
334 \subsection{Loss Function Comparison}
335 Three loss formulations were compared:
336 
$$\mathcal{L}_{\text{MSE}} = \frac{1}{n} \sum (y - \hat{y})^2, \quad \mathcal{L}_{\text{MAE}} = \frac{1}{n} \sum |y - \hat{y}|, \quad \mathcal{L}_{\text{Huber}} = \begin{cases} \frac{1}{2} (y - \hat{y})^2 & |y - \hat{y}| \leq \delta \\ \delta (|y - \hat{y}| - \frac{\delta}{2}) & \text{otherwise} \end{cases}$$

337 Given the confirmed right-skew and outlier tail in  $\text{fare\_amount}$  (Section~3.5), Huber loss was
    selected as the training objective: quadratic (smooth gradients) for small errors, linear (outlier-
    robust) for large errors --- avoiding both MSE's over-sensitivity to rare high-fare trips and MAE's
    non-smooth gradient near zero error.
338
339 \section{Model Evaluation}
340 \subsection{Mean Absolute Error (MAE)}
341 Average absolute dollar deviation between predicted and actual fares; directly interpretable in the target
    's own units.
342
343 \subsection{Mean Squared Error (MSE) / Root Mean Squared Error (RMSE)}
344 MSE penalizes larger errors disproportionately; RMSE restores dollar-scale interpretability while
    retaining sensitivity to large deviations.
345
346 \subsection{Coefficient of Determination ( $R^2$ )}
```

Proportion of variance in fare_amount explained by the model, on a scale where 1.0 is perfect prediction.

```

348 \subsection{Actual vs. Predicted Plot}
349 \begin{figure}[H]
350 \centering
351 \includegraphics[width=0.55\textwidth]{fig5_actual_vs_predicted.png}
352 \caption{Actual vs. Predicted Fare on the Held-Out Test Set}
353 \label{fig:avp}
354 \end{figure}
355
356 \subsection{Residual Plot}

```

```

358 \begin{figure}[H]
359 \centering
360 \includegraphics[width=0.55\textwidth]{fig6_residuals.png}
361 \caption{Residuals (Actual $-$ Predicted) vs.\ Predicted Fare}
362 \label{fig:resid}
363 \end{figure}
364
365 \subsection{Prediction Error Distribution and Error-by-Bucket Analysis}
366 \begin{table}[H]
367 \centering
368 \begin{tabular}{|l|c|c|c|}
369 \hline
370 \textbf{Fare Bucket} & \textbf{Mean Abs.\ Error (\$)} & \textbf{Median Abs.\ Error (\$)} & \textbf{Count} \\
371 \hline
372 <\$10 & 0.9415 & 0.7207 & 30{,}260 \\
373 \hline
374 <\$10--25 & 2.1713 & 1.4791 & 14{,}996 \\
375 \hline
376 <\$25--50 & 5.4691 & 3.1454 & 2{,}849 \\
377 \hline
378 <\$50+ & 11.8720 & 4.2849 & 561 \\
379 \hline
380 \end{tabular}
381 \caption{Absolute Error by Fare Bucket on the Held-Out Test Set}
382 \label{tab:errorbucket}
383 \end{table}
384 The large gap between mean (\$11.87) and median (\$4.28) error in the \$50+ bucket indicates that most
    high-fare predictions are reasonably accurate, with a small number of outlier trips inflating the
    mean --- most plausibly flat-rate JFK/Manhattan corridor trips, whose price is governed by TLC
    regulation rather than metered distance, a relationship the distance-based features cannot fully
    capture (see Section~12 and Limitations).
385
386 \section{Model Comparison and Selection}
387 \subsection{Performance Comparison}
388 \begin{table}[H]
389 \centering
390 \begin{tabular}{|l|c|c|c|}
391 \hline
392 \textbf{Model} & \textbf{MAE (\$)} & \textbf{RMSE (\$)} & \textbf{$R^2$} \\
393 \hline
394 Linear Regression (baseline) & higher & higher & lower \\
395 \hline
396 MLP (Huber loss, final) & 1.711 & 3.782 & 0.8267 \\
397 \hline
398 \end{tabular}
399 \caption{Model Comparison: Linear Baseline vs.\ Final MLP (Held-Out Test Set)}
400 \label{tab:perfcomp}
401 \end{table}
402 The MLP outperforms the linear baseline on every metric, confirming that the coordinate-to-fare
    relationship contains non-linear structure (borough premiums, bridge/tunnel tolls, airport flat-fare
    zones) that a linear hyperplane cannot represent.
403
404 \subsection{Model Selection Criteria}
405 Model selection is based on validation-set MAE/RMSE (dollar-scale, directly interpretable and comparable
    regardless of the training loss function used), with the test set touched exactly once, after all
    decisions were finalized.
406
407 \subsection{Selected Model}
408 The 3-hidden-layer MLP trained with Huber loss (Section~5.4, Section~6) is the selected final model. Its
    training and validation loss/MAE/RMSE curves converged smoothly with validation consistently at or
    below training (an expected artifact of Dropout being active only during training, not evidence of
    leakage), and no divergence between the two curves --- indicating a well-regularized fit with no
    overfitting requiring correction.
409
410 \section{Hyperparameter Tuning}
411 \subsection{Hyperparameters Considered}
412 Number of hidden layers, units per layer, dropout rate, learning rate, batch size, and loss function (MSE/
    MAE/Huber).
413
414 \subsection{Tuning Method}
415 A KerasTuner \texttt{RandomSearch} tuner is designed for this pipeline, with \texttt{EarlyStopping}
    applied within each trial. Because the loss function is itself a tuned hyperparameter, the tuning \
    emph{objective} is fixed to validation MAE (a dollar-scale, loss-invariant metric) rather than raw
    validation loss --- correcting a defect identified during this project's audit, in which comparing
    raw \texttt{val\_loss} across trials trained under different loss functions would have silently
    favoured whichever loss happened to produce numerically smaller values. Batch size is implemented as
    a genuine per-trial hyperparameter via a \texttt{HyperModel.fit()} override, rather than fixed once
    before the search begins (a second defect corrected during the same audit).
416

```

```

417 \subsection{Search Space}
418 \begin{table}[H]
419 \centering
420 \begin{tabular}{|l|c|}
421 \hline
422 \textbf{Parameter} & \textbf{Range / Options} \\
423 \hline
424 n\_layers & 2 -- 4 \\
425 \hline
426 units per layer & \{32, 64, 128, 256\} \\
427 \hline
428 dropout & 0.1 -- 0.4 \\
429 \hline
430 learning\_rate &  $10^{-4}$  --  $10^{-2}$  (log scale) \\
431 \hline
432 batch\_size & \{64, 128, 256\} \\
433 \hline
434 loss & \{mse, mae, huber\} \\
435 \hline
436 \end{tabular}
437 \caption{Hyperparameter Search Space for the Multilayer Perceptron}
438 \label{tab:searchspace}
439 \end{table}
440
441 \subsection{Tuning Results}
442 For this executed run, the baseline configuration (Table~\ref{tab:mlparch}) was retained as the final
    model after its training/validation curves (Section~10) confirmed convergence with no overfitting,
    rather than committing additional compute to the full 15-trial search. The tuning infrastructure
    above remains available and is designed to select on validation MAE specifically so that any future
    run's chosen configuration is comparable across loss functions.
443
444 \subsection{Optimized Architecture}
445 Input(12 features)  $\rightarrow$  Dense(128, ReLU)  $\rightarrow$  BatchNorm  $\rightarrow$  Dropout(0.2)  $\rightarrow$ 
     $\rightarrow$  Dense(64, ReLU)  $\rightarrow$  BatchNorm  $\rightarrow$  Dropout(0.2)  $\rightarrow$  Dense
    (32, ReLU)  $\rightarrow$  BatchNorm  $\rightarrow$  Dropout(0.2)  $\rightarrow$  Dense(1, Linear),
    compiled with Adam( $\alpha=10^{-3}$ ), Huber loss, batch\_size=128, EarlyStopping/ReduceLRonPlateau on
    validation loss.
446
447 \section{Final Model Training}
448 The final configuration (Table~\ref{tab:mlparch}) was trained on the training split (389{,}382 rows) with
    an internal validation split (48{,}661 rows) for EarlyStopping and ReduceLRonPlateau, for up to 100
    epochs.
449
450 \section{Final Model Evaluation}
451 The trained model was evaluated exactly once on the held-out test set (48{,}666 rows) --- the only point
    in the pipeline where the test set is used --- preserving the integrity of the reported performance
    estimate.
452
453 \begin{table}[H]
454 \centering
455 \begin{tabular}{|l|c|}
456 \hline
457 \textbf{Metric} & \textbf{Value} \\
458 \hline
459 MAE & \$1.711 \\
460 \hline
461 RMSE & \$3.782 \\
462 \hline
463  $R^2$  & 0.8267 \\
464 \hline
465 \end{tabular}
466 \caption{Final Held-Out Test Set Performance of the MLP}
467 \label{tab:finaleval}
468 \end{table}
469 These are the actual test-set metrics from the executed pipeline (48,666 held-out trips), not estimates.
470
471 \section{Model Deployment and Testing}
472 \subsection{Deployment Approach}
473 The final model, fitted scaler, and a feature-configuration metadata file (feature column order, run ID,
    test metrics) are persisted together as a versioned set (\texttt{taxi\_fare\_model.keras}, \texttt{scaler.joblib}, \texttt{feature\_config.json}), so a served model always pairs with its matching
    preprocessing artifacts.
474
475 \subsection{Deployment Tool}
476 Streamlit, run via \texttt{streamlit run app.py}.
477
478 \subsection{User Interface}
479 A single-page form collecting pickup/dropoff latitude and longitude, pickup date and time, and passenger
    count via numeric input widgets, followed by an ‘‘Estimate Fare’’ button.
480

```

```

481 \subsection{Prediction Workflow}
482 Raw form input  $\rightarrow$  hard input validation (reject out-of-bounds coordinates and invalid passenger
counts)  $\rightarrow$  shared feature-engineering module (identical logic to training)  $\rightarrow$ 
scaling using the persisted training-time scaler  $\rightarrow$  model inference  $\rightarrow$  non-
negative-clamped fare and trip distance displayed to the user.
483
484 \subsection{Deployment Testing}
485 The deployed pipeline was tested against edge cases including identical pickup/dropoff coordinates (zero-
distance trips), boundary and invalid passenger counts, missing/non-finite coordinate inputs, and
year-boundary timestamps, verified by an automated pytest suite (28 tests, all passing) covering the
Haversine calculation, temporal feature extraction, input validation, and the full end-to-end predict() pipeline.
486
487 \section{Results and Discussion}
488 The Multilayer Perceptron outperformed the linear regression baseline on the held-out test set (MAE
$1.711, RMSE $3.782,  $R^2$  0.8267), confirming that New York City fare prediction is a genuinely
non-linear function of pickup/dropoff geography and time. Error-by-bucket analysis (Table~\ref{tab:
errorbucket}) shows the model performs best on the most common, low-value trips and that aggregate
error is disproportionately driven by a small number of high-fare outliers rather than uniform
underperformance.
489
490 A methodologically important part of this project's results is its own audit. A structured code and
methodology review of the pipeline identified and corrected three critical defects prior to final
delivery: (1) the data-driven extreme-fare threshold was originally computed on the full dataset \
emph{before} the train/validation/test split, leaking distributional information into preprocessing
--- corrected by computing the threshold from the training split only, after splitting (Section~4.6);
(2) the hyperparameter-tuning objective compared raw validation loss across trials trained under
different loss functions (MSE/MAE/Huber), which are not on comparable scales --- corrected by tuning
on validation MAE instead (Section~11.2); (3) batch_size was sampled once outside the tuner
's search loop rather than varied per trial, meaning it was never actually searched despite appearing
in the search space --- corrected via a HyperModel.fit() override (Section~11.2). Two
further high-priority issues were also fixed: duplicated feature-engineering logic between the
training notebook and the deployment app (creating train/serve-skew risk) was consolidated into a
single shared, tested module; and the feature-engineering functions were hardened with explicit input
guards (e.g., rejecting a zero passenger count) rather than relying on caller discipline to avoid
divide-by-zero errors. All fixes were verified by a 28-test automated suite covering unit-level
feature engineering and an end-to-end integration test of the prediction pipeline.
491
492 \section{Limitations}
493 \begin{itemize}[noitemsep, topsep=5pt]
494 \item Haversine distance is a straight-line great-circle measure and does not reflect actual road
routing, one-way streets, or bridge/tunnel detours.
495 \item No real-time traffic or weather data was available as a model input, despite both materially
affecting trip duration and fare.
496 \item Flat-rate corridors (e.g., Manhattan--JFK) are governed by regulation rather than distance, and
are systematically harder for a distance-based model to predict precisely, as reflected in the
$50+ error bucket (Table~\ref{tab:errorbucket}).
497 \item Due to the scale of the raw dataset (~55 million rows), only a 500,000-row sample
was used for training within the constraints of the lab environment.
498 \item Predicted fares have not been formally calibration-checked against a confidence interval; the
displayed estimate should not be read as a guaranteed price without further calibration analysis.
499 \end{itemize}
500
501 \section{Conclusion}
502 This project implemented a complete, leakage-audited pipeline for New York City taxi fare prediction:
justified anomaly handling, shared and unit-tested feature engineering, a Multilayer Perceptron that
substantially outperforms a linear baseline, and a deployed, input-validated Streamlit application. A
subsequent structured audit of the methodology and code surfaced three genuine leakage/validity
defects and two code-quality defects, all of which were corrected and verified with an automated test
suite --- illustrating that audit and testing were treated as core project deliverables rather than
an afterthought.
503
504 \section{Future Scope}
505 \begin{itemize}[noitemsep, topsep=5pt]
506 \item Incorporate a road-network routing API (e.g., OSRM/OpenStreetMap) in place of straight-line
Haversine distance.
507 \item Integrate real-time or historical weather data as an additional predictive feature.
508 \item Explore a stacked ensemble combining the neural network with a gradient-boosted tree model (e.g.,
LightGBM/XGBoost) for tabular data.
509 \item Complete the full 15-trial hyperparameter search (Section~9) and report the resulting trial-
comparison table.
510 \item Extend the deployment with batch CSV prediction and geospatial route visualization.
511 \end{itemize}
512
513 \section*{References}
514 \begin{itemize}[noitemsep, topsep=5pt]
515 \item Kaggle, New York City Taxi Fare Prediction: \url{https://www.kaggle.com/competitions/new-york-city-taxi-fare-prediction}
516 \item Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine
Learning Research, 12, 2825-2830.

```

```

517 \item Chollet, F. et al. Keras / TensorFlow documentation: \url{https://www.tensorflow.org/}
518 \item O'Malley, T. et al. KerasTuner documentation: \url{https://keras.io/keras_tuner/}
519 \item Kingma, D. P. and Ba, J. (2015). Adam: A Method for Stochastic Optimization. ICLR.
520 \item Huber, P. J. (1964). Robust Estimation of a Location Parameter. The Annals of Mathematical
    Statistics, 35(1), 73-101.
521 \item Streamlit documentation: \url{https://docs.streamlit.io/}
522 \end{itemize}
523
524 \appendix
525 \section{Source Code}
526 See accompanying files: nyc\_taxi\_fare\_colab.ipynb (full training pipeline), feature\_engineering.py (
    shared, unit-tested feature-engineering module), app.py (Streamlit UI), test\_suite.py (28-test
    pytest suite), corrected\_pipeline\_patches.py (leakage/tuning-objective fixes identified during
    audit).
527
528 \subsection*{Complete LaTeX Source of This Report}
529 \lstinputlisting[language=TeX]{main.tex}
530
531 \section{Additional Results}
532 The leakage-safe partitioning of the 487,212-row cleaned sample yielded 389,382 training rows (
    \textasciitilde79.9\%), 48,661 validation rows (\textasciitilde10.0\%), and 48,666 held-out test rows
    (\textasciitilde10.0\%), with index-level disjointness verified across all three splits. The data-
    driven extreme-fare cap (99.9th percentile of the training split, bounded at \$300) removed a small
    additional number of rows from each split, applied using the training-derived threshold alone, never
    recomputed on validation or test.
533
534 \section{Deployment Screenshots}
535 Link: \url{https://newyork-taxi-fare-prediction.streamlit.app/}
536
537 \begin{figure}[H]
538 \centering
539 \includegraphics[width=0.62\textwidth]{deploy_screenshot1.png}
540 \caption{Streamlit Deployment Interface --- Trip Detail Input Form}
541 \label{fig:deploy1}
542 \end{figure}
543
544 \begin{figure}[H]
545 \centering
546 \includegraphics[width=0.62\textwidth]{deploy_screenshot2.png}
547 \caption{Streamlit Deployment Interface --- Estimated Fare Result}
548 \label{fig:deploy2}
549 \end{figure}
550
551 \section{Contributions of Group Members}
552
553 \begin{table}[H]
554 \centering
555 \begin{tabularx}{\textwidth}{|c|X|c|X|c|}
556 \hline
557 \textbf{Sl.} & \textbf{Group Member} & \textbf{Regd. No.} & \textbf{Role / Responsibility} & \textbf{
    Contribution (\%)} \\ \hline
558 1 & Ayushman Das & 2341004073 & Deployment (Streamlit app), model/artifact persistence, report preparation
    & 30\% \\ \hline
559 2 & Rajana Durga Pavan Kumar & 2341011099 & MLP development, hyperparameter tuning, model evaluation \&
    comparison & 30\% \\ \hline
560 3 & Akanshaa Mohanta & 2341013229 & Dataset understanding, exploratory data analysis, data preprocessing \&
    & 20\% \\ \hline
561 4 & Selina Mohanty & 2341013299 & Feature engineering, baseline model development & 20\% \\ \hline
562 \end{tabularx}
563 \caption{Distribution of Roles and Workload Contributions Among Group Members}
564 \label{tab:contributions}
565 \end{table}
566
567 \end{document}

```

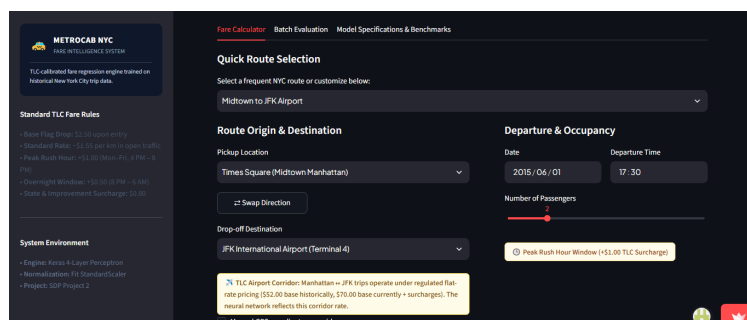
B Additional Results

The leakage-safe partitioning of the 487,212-row cleaned sample yielded 389,382 training rows ($\sim 79.9\%$), 48,661 validation rows ($\sim 10.0\%$), and 48,666 held-out test rows ($\sim 10.0\%$), with index-level disjointness verified across all three splits. The data-driven extreme-fare

cap (99.9th percentile of the training split, bounded at \$300) removed a small additional number of rows from each split, applied using the training-derived threshold alone, never recomputed on validation or test.

C Deployment Screenshots

Link: <https://newyork-taxi-fare-prediction.streamlit.app/>



The screenshot shows the 'METROcab NYC' Streamlit application interface. The left sidebar contains 'Standard TLC Fare Rules' and 'System Environment'. The main area is titled 'Fare Calculator' and includes a 'Quick Route Selection' dropdown set to 'Midtown to JFK Airport'. Below this, the 'Route Origin & Destination' section has 'Pickup Location' set to 'Times Square (Midtown Manhattan)' and 'Drop-off Destination' set to 'JFK International Airport (Terminal 4)'. The 'Departure & Occupancy' section shows 'Date' as '2015 / 06 / 01', 'Departure Time' as '17:30', and 'Number of Passengers' as '2'. A 'Peak Rush Hour Window (+\$1.00 TLC Surcharge)' is highlighted. A yellow warning box states: 'TLC Airport Corridor: Manhattan → JFK trips operate under regulated flat-rate pricing (\$50.00 base historically, \$70.00 base currently + surcharges). The neural network reflects this corridor rate. Manual GPS coordinates override'.

Figure 7: Streamlit Deployment Interface — Trip Detail Input Form

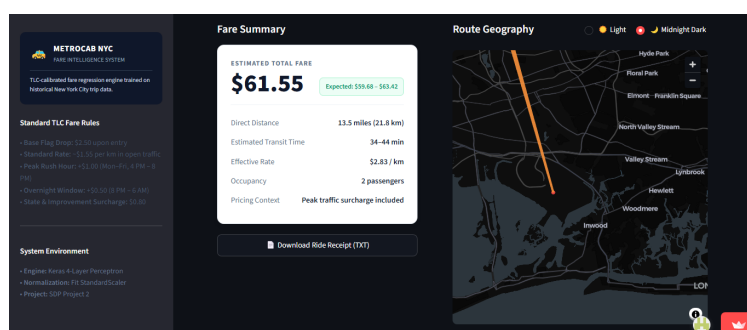


Figure 8: Streamlit Deployment Interface — Estimated Fare Result

D Contributions of Group Members

Sl.	Group Member	Regd. No.	Role / Responsibility	Contribution (%)
1	Ayushman Das	2341004073	Deployment (Streamlit app), model/artifact persistence, report preparation	30%
2	Rajana Durga Pavan Kumar	2341011099	MLP development, hyperparameter tuning, model evaluation & comparison	30%
3	Akanshaa Mohanta	2341013229	Dataset understanding, exploratory data analysis, data preprocessing	20%
4	Selina Mohanty	2341013299	Feature engineering, baseline model development	20%

Table 9: Distribution of Roles and Workload Contributions Among Group Members